

Informe Tarea 1: Programación Modular y Análisis de Rendimiento

Integrantes: Lucianno Lopez, Benjamín Jofre, Eloy Saldoval, Franco Valdivia, Astrid Barrios

Institución: Universidad Técnica Federico Santa María

Fecha: 9 de mayo de 2026

Parte 1: Modularización y Uso de Enumeraciones

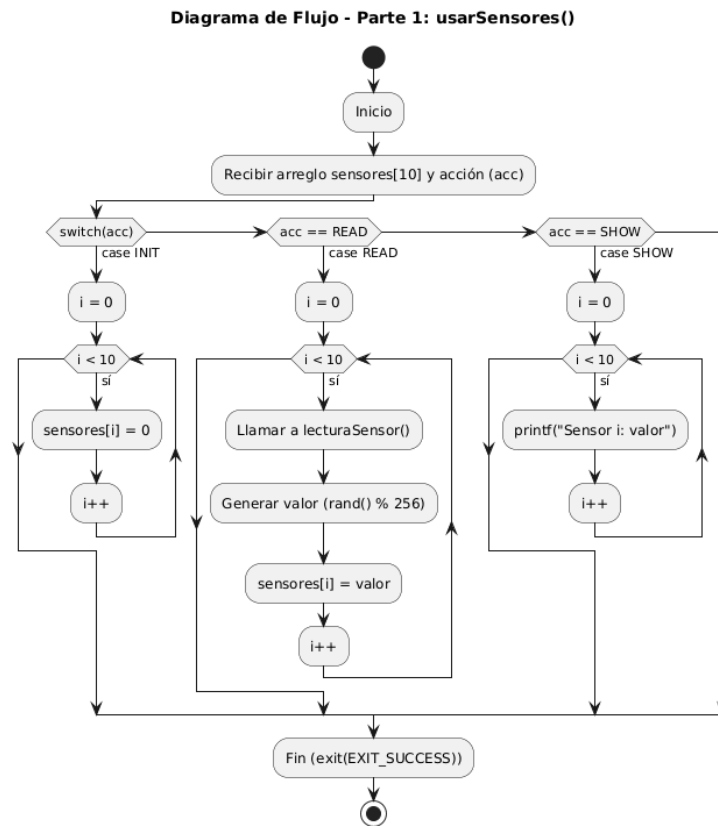
1.1 Diseño de la función lecturaSensor()

El diseño de lecturaSensor() se enfoca en la simulación de adquisición de datos. Sus requisitos operacionales incluyen la generación de un valor entero aleatorio que represente una medición en milivoltios. Para asegurar la variabilidad en cada ejecución, se utiliza la semilla de tiempo en el main.

1.2 Justificación del enum Accion

La implementación del enumerador Accion (con valores como INIT, READ, SHOW) es fundamental para la **legibilidad y mantenibilidad** del código. En lugar de utilizar "números mágicos" (0, 1, 2) que carecen de significado semántico, el enum permite que la función usarSensores sea autodocumentada, facilitando la comprensión de qué operación se está realizando sobre el arreglo de sensores en cada llamada.

1.3 Explicación del Diseño (Diagrama de Flujo)



Explicación del proceso:

1. **Inicio:** Se inicializa la semilla aleatoria `srand(time())` y se declara el arreglo de 10 sensores.
2. **Llamada Modular:** Se invoca a `usarSensores(acc)` donde `acc` determina el flujo mediante un switch.
3. **Casos:**
 - **INIT:** Recorre el arreglo asignando 0 a cada posición.
 - **READ:** Llena cada posición con el retorno de `lecturaSensor()`, el cual genera un valor mediante `rand() % 256`.
 - **SHOW:** Imprime en consola el valor de cada sensor.
4. **Fin:** El programa finaliza con `EXIT_SUCCESS`.

Parte 2: Análisis de Tiempos de Ejecución y E/S

2.1 Hito 1: Análisis de cicloEjemplo() e Instrumentación

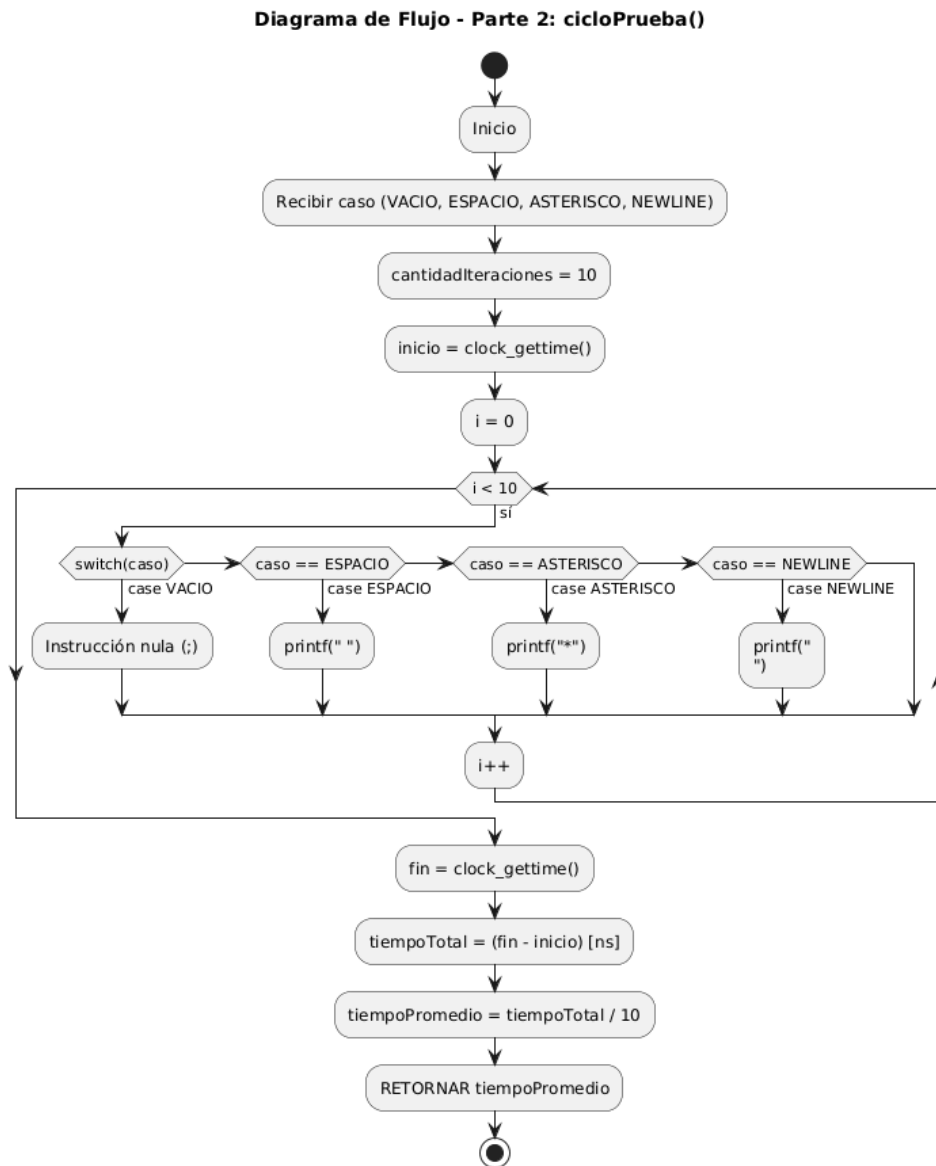
A continuación, se responden las preguntas planteadas sobre la medición de tiempos:

- **¿Qué retorna clock()?:** Retorna el número de pulsos de reloj (ticks) consumidos por el programa desde su inicio.
- **Unidades de inicio y fin:** Estas variables están en unidades de clock_t (ciclos de reloj).
- **División por CLOCKS_PER_SEC:** Se realiza para convertir los ciclos de reloj a segundos, ya que esta constante define cuántos "ticks" ocurren en un segundo real según el sistema.
- **Prefijo (double):** Es un **cast** o conversión de tipo explícita. Sirve para asegurar que la operación aritmética se realice en punto flotante, evitando la pérdida de decimales por truncamiento de división entera.

2.2 Hito 2: Diseño de cicloPrueba()

La función `cicloPrueba(caso_t caso)` es el núcleo del experimento de la Parte 2, diseñada para cuantificar de manera aislada el impacto que tienen las operaciones de Entrada/Salida (E/S) frente al procesamiento puro en memoria.

A continuación, el diagrama de flujo:



Explicación del proceso:

La función recibe un parámetro caso de tipo caso_t. Se capturan las marcas de tiempo antes y después de un ciclo de 10 iteraciones. Dentro del ciclo, un switch ejecuta la instrucción correspondiente: una instrucción nula (;) para VACIO, o una llamada a printf con distintos caracteres. El tiempo total se calcula en nanosegundos y se divide por la cantidad de iteraciones para obtener el promedio unitario.

2.3 Hito 3: Resultados y Análisis

A partir de las 10 mediciones realizadas, se obtuvieron los siguientes valores promedio:

CASO	tprom 1 [ns]	tprom 2 [ns]	tprom 3 [ns]	tprom 4 [ns]	tprom 5 [ns]	tprom 6 [ns]	tprom 7 [ns]	tprom 8 [ns]	tprom 9 [ns]	tprom 10 [ns]	Promedio	Desv. Estandar
NADA	1,5	1,55	1,54	1,52	1,55	1,53	1,51	1,55	1,95	1,61	1,581	0,12628935
ESPACIO	27700	27300	34300	29200	26900	27700	33300	33300	28200	29400	29730	2669,475604
ASTERISCO	32800	32700	27800	31200	27700	34200	27500	28000	27800	32900	30260	2591,601821
NEWLINE	24600	25600	28000	27500	33200	25800	26300	27100	26500	26300	27090	2236,269215

Análisis de resultados:

Existe una diferencia de magnitud abismal (de aproximadamente **18.800 veces**) entre el caso NADA y cualquier operación que involucre printf.

1. **Costo de E/S:** El caso NADA (~1,58 ns) refleja el tiempo que le toma al procesador simplemente incrementar un contador y evaluar una condición de salto en memoria/registros.
2. **Interrupción del Sistema:** Las operaciones de ESPACIO, ASTERISCO y NEWLINE requieren llamadas al sistema (syscalls) para interactuar con el hardware de salida (terminal). Esto implica un cambio de contexto del CPU, lo que dispara el tiempo a los ~27.000 - 30.000 ns.
3. **Conclusión:** Se evidencia que la Entrada/Salida (E/S) es el cuello de botella principal en el rendimiento de un software, siendo órdenes de magnitud más lenta que el procesamiento en memoria.